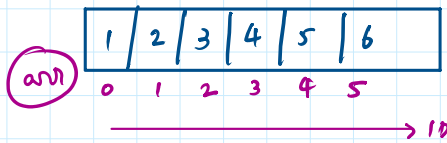
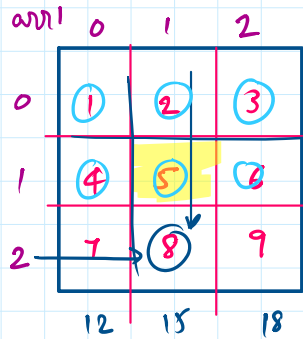
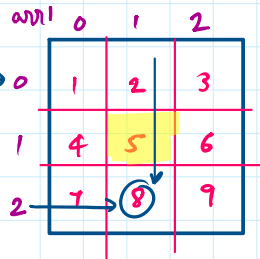


2D Arrays

2-dimensional, rows, cols.



→ 2D Array →



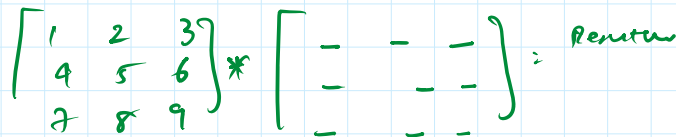
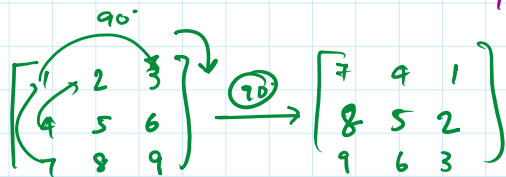
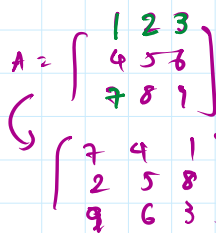
sum = total sum

row

max = 24

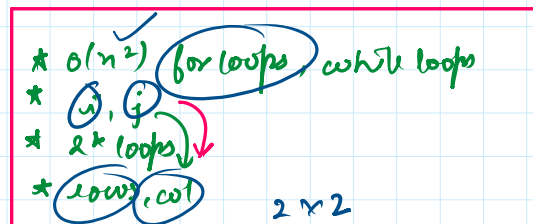
cout << arr[i][j] << endl;

(5)
arr[2][1] = 8



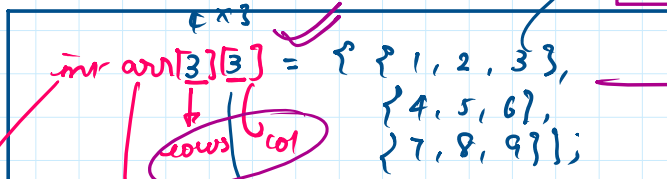
→ 2D Array &

8 n' queen



2x2
3x4
4x3

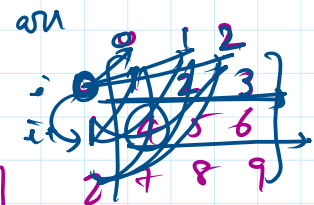
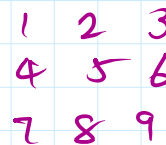
values that you store in 2D.



data type

name of var

size of arr (2D)



// code

$O(n^2)$ ←

```
void 2DArray (int arr[3][3]) {
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            cout << arr[i][j] << " ";
        }
    }
}
```

2D matrix

2D array.

$O(n^2)$

for (for cout << arr[i][j] << " ");

int i = 2;
int * p = &i;
int * q =

```

for (int j = 0; j < 3; j++) {
    (int) > arr[i][j]) {
        // swap / print
    }
}

```

for (for
arr)

count arr[i][j]

0	0	1
0	1	2
0	2	3
1	0	4
1	1	5
1	2	6
2	2	9

0	1	2	3
1	4	5	6
2	7	8	9

key:
i, j → value / index / position

(2, 2) code

```

for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        if (arr[i][j] == key) {
            count(i, j)++;
        }

```

count(i, j)++

code

0	1	2	3
1	4	5	6
2	7	8	9

maximum

6 15 24
sum
count

```

for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        sum = sum + arr[i][j];
count(sum);

```

sum 15

15 count

24

0 + 1
1 + 2
3 + 3
0 + 4
4 + 5
9 + 6

j = 0 f = 5

0	1	2
1	4	5
2	7	8

15
15
30

main diagonal

i == j

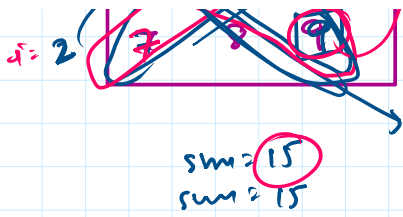
0	0	1
1	1	5
2	2	9

sum = 15

```

if (i == j) {
    sum += arr[i][j];
}
count(sum);

```



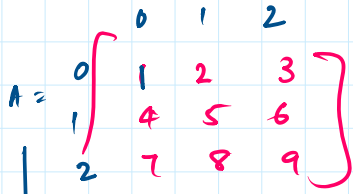
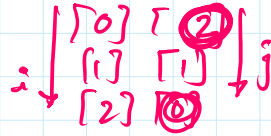
30%

$2 \times 2 \rightarrow 19$
sum = 15

count < sum;

$i = j$

for (int i = 0; i < n; i++)
for (int j = 0; j < n; j++)

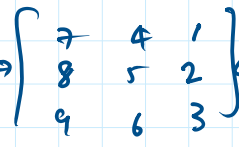
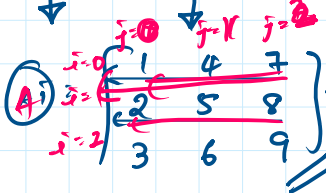


swap (row, col $\rightarrow$ col, row)
 $\rightarrow A^T$

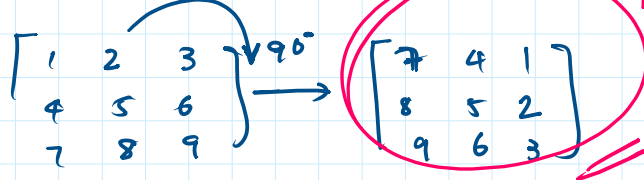


$arr[i][j]$

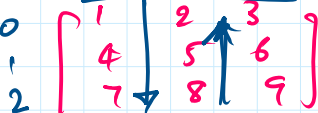
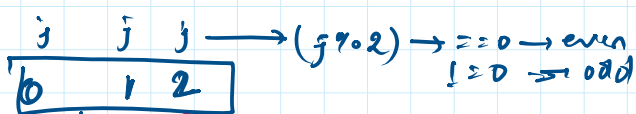
(90°) clockwise



(90°)



- 1) A^T
- 2) reverse it by row



$\rightarrow op$

- 1
- 4
- 7
- 8
- 5
- 2
- 3
- 6
- 9

even
op
odd
op

① values, reference, that store the value of address of other variable.

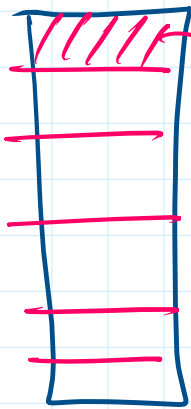
→ C, C++ → free ptr delete both

→ (java, python) → memory manage
garbage collection (GC) &

→ "pointer" C++, C

② C → C++ → "delete"

C, C++

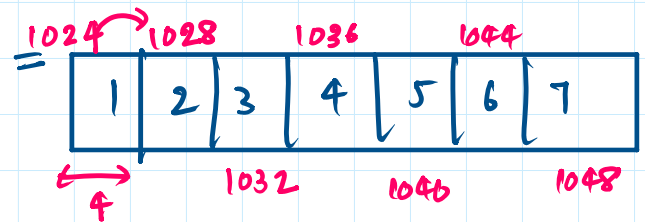
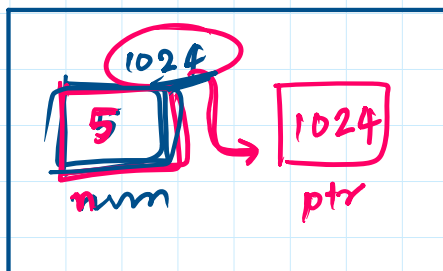


heap

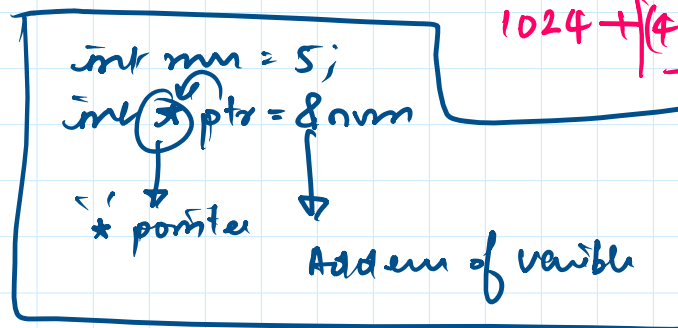
③ "new int[3]" → dynamic

"realloc" → delete an delete val;

java → "JVM" → pointer,
python



pointer =



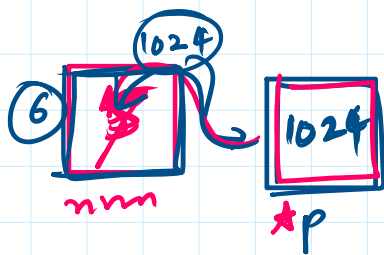
$1024 + (4) \text{ size} = \text{address}$

int num = 5;
int *p = #

1024

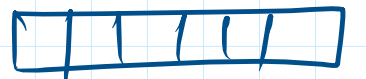
1024 + 4 = 1028

<1



$1024 \xrightarrow{+4} 1028 \rightarrow \dots \rightarrow 10$

$num = num + 1 \rightarrow 6$
 $cout \ll p \rightarrow 1024 \rightarrow \text{confused}$ *→ Here a value.*

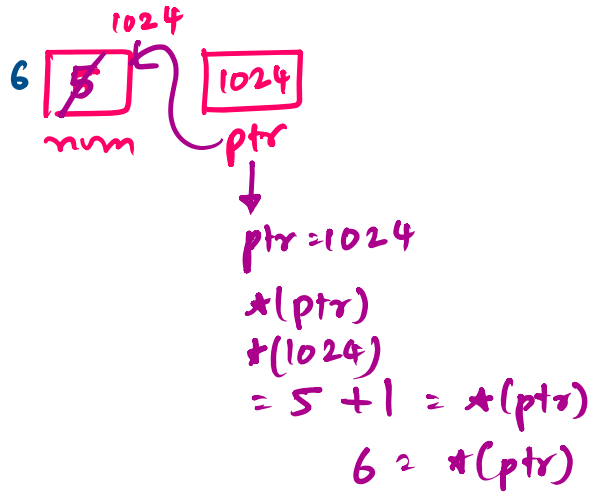

 $10 \times 4 = 40 \text{ bytes.}$

$cout \ll *p$
 $\downarrow$
 $\frac{* (1024)}{5}$

$cout \ll (*p)++$
 $++(*p)$
 $\downarrow$
 $\frac{* (1024)}{+ (5)} \rightarrow 6$

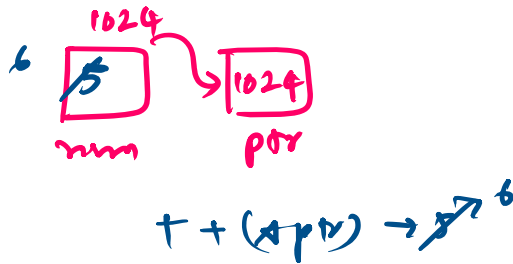
```
main.cpp
1 #include<iostream>
2 using namespace std;
3 int main(){
4     int num = 5;
5     int *ptr = &num;
6     *ptr = *ptr + 1;
7     cout<<*ptr<<endl;
8     return 0;
9 }
```

Handwritten notes: $5 \rightarrow 6$, $ptr = 1024$, $6 = *ptr$



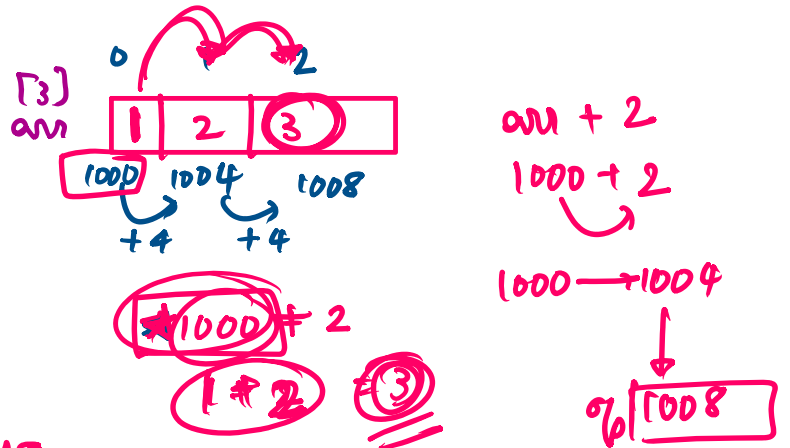
```
main.cpp
1 #include<iostream>
2 using namespace std;
3 int main(){
4     int num = 5;
5     int *ptr = &num;
6     cout<<*ptr<<endl;
7     cout<<*(ptr)<<endl;
8     return 0;
9 }
```

Handwritten notes: $ptr = 1024$, $*(ptr) \rightarrow 6$



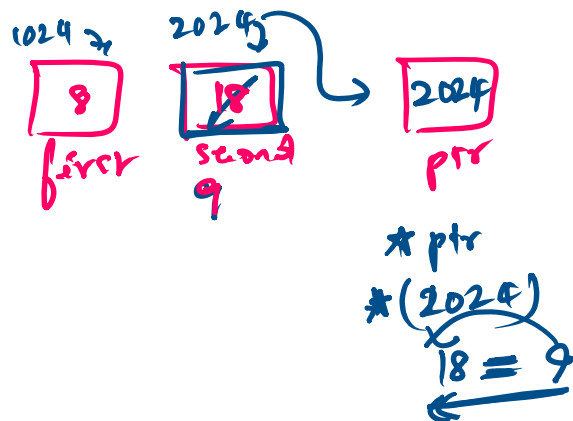
```
main.cpp
1 #include<iostream>
2 using namespace std;
3 int main(){
4     int arr[3] = {1,2,3};
5     cout<<*arr+2<<endl;
6     return 0;
7 }
```

Handwritten notes: $arr = 1000$, $arr + 2 \rightarrow 1008$



```
main.cpp
1 #include<iostream>
2 using namespace std;
3 int main(){
4     int first = 8;
5     int second = 18;
6     int *ptr = &second;
7     *ptr = 9;
8     cout<<first<<" "<<second<<endl;
9 }
```

Handwritten notes: $first = 8$, $second = 18$, $ptr = 2024$, $*ptr = 9$, $18 = 9$



Q-17 @ $\star$ segmentation fault

(Q-17) @

★ segmentation fault

```
main.cpp
1 #include<iostream>
2 using namespace std;
3 int main(){
4     int *p = 0;
5     int first = 110;
6     *p = first;
7     cout<<*p<<endl;
8 }
```



arr
1008

$\&arr[0] \rightarrow 1000$

$arr \rightarrow 1000$

$arr[0] \rightarrow 1000$



$*p = 110$

